Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Лабораторна робота № 1

з дисципліни «Протоколи та алгоритми електронного голосування »

Виконав: студент групи ІП-34 Туболев М. Р.

Перевірив: Асистент кафедри, PhD Нестерук А. О.

2026

1. Тема, мета та завдання роботи

Тема: простий протокол Е-голосування.

Мета роботи: програмно реалізувати всі кроки простого протоколу електронного голосування, у якому беруть участь два кандидати та п’ять виборців, використовуючи алгоритм RSA як для шифрування бюлетенів, так і для формування та перевірки електронного цифрового підпису (ЕЦП).

Завдання роботи:

- реалізувати самостійну генерацію кожним виборцем власної пари ключів RSA для формування та перевірки ЕЦП;
- реалізувати формування виборчою комісією (ВК) списку виборців із зазначенням їх ПІБ та відповідних відкритих ключів, а також списку кандидатів;
- реалізувати генерацію ВК власної пари ключів RSA для шифрування бюлетенів і публікацію відкритого ключа;
- реалізувати формування виборцем електронного бюлетеня, його підписання та шифрування відкритим ключем ВК;
- реалізувати розшифрування отриманого бюлетеня ВК за допомогою власного закритого ключа;
- реалізувати перевірку наявності виборця у списку, перевірку ЕЦП заздалегідь закріпленим відкритим ключем та перевірку повторного голосування;
- реалізувати прийняття або відхилення бюлетеня за результатами перевірок і фіксацію факту голосування;
- реалізувати підрахунок прийнятих голосів, публікацію результатів та врахувати можливість рівномірного розподілу голосів між кандидатами;
- для кожної перевірки показати як позитивний, так і негативний сценарій.

2. Короткий опис програмної реалізації

Протокол реалізовано мовою TypeScript і виконується середовищем Node.js без етапу збірки. Криптографічні бібліотеки не використовувалися: модульна арифметика, генерація простих чисел, розширений алгоритм Евкліда, блокове шифрування RSA, хеш квадратичної згортки та ЕЦП написані вручну, щоб формули збігалися з методичними вказівками. Усі великі числа представлені вбудованим типом bigint.

Модуль	**Призначення**
src/bigint-math.ts	модульне піднесення до степеня, розширений алгоритм Евкліда, обернене за модулем, тест простоти Міллера — Рабіна, генерація простих чисел
src/rsa.ts	генерація пари ключів, блокове шифрування та розшифрування, хеш квадратичної згортки, формування і перевірка ЕЦП
src/types.ts	структури бюлетеня, конверта, запису у списку виборців, причини відхилення, канонічна серіалізація бюлетеня
src/voter.ts	клас виборця: власні ключі, заповнення, підписання та шифрування бюлетеня
src/commission.ts	клас виборчої комісії: списки виборців і кандидатів, розшифрування, перевірки, підрахунок голосів
src/main.ts	демонстраційний сценарій: позитивні та негативні випадки, нічия, зведення властивостей протоколу
tests/	21 модульний тест (вбудований тест-ранер node:test)

Таблиця 1 — Структура програмної реалізації

2.1. Генерація ключів RSA

Для кожного учасника обираються два різні прості числа p і q довжиною 256 біт кожне, обчислюється модуль n = p·q та функція Ейлера φ(n) = (p−1)(q−1). Відкритий показник e обирається непарним і взаємно простим з φ(n) (за замовчуванням 65537), а закритий показник d обчислюється як число, обернене до e за модулем φ(n), за допомогою розширеного алгоритму Евкліда.

```
/**
 * Генерація пари ключів RSA.
 * p, q — різні прості; n = p*q; φ(n) = (p-1)(q-1);
 * e — непарне, 1 < e < φ(n), НСД(e, φ(n)) = 1; d = e⁻¹ mod φ(n).
 */
export function generateKeyPair(bits = 512): KeyPair {
  if (bits < 64) throw new RangeError("Для блокового шифрування потрібно
щонайменше 64 біти");

  const halfBits = Math.floor(bits / 2);
```

```
  let p = generatePrime(halfBits);
  let q = generatePrime(bits - halfBits);
  while (p === q) q = generatePrime(bits - halfBits);

  const n = p * q;
  const phi = (p - 1n) * (q - 1n);

  const e = gcd(DEFAULT_PUBLIC_EXPONENT, phi) === 1n
    ? DEFAULT_PUBLIC_EXPONENT
    : findCoprimeExponent(phi);
  const d = modInverse(e, phi); // розширений алгоритм Евкліда

  return { publicKey: { e, n }, privateKey: { d, n }, parameters: { p, q, phi }
};
}
```

Лістинг 1 — Генерація пари ключів (src/rsa.ts)

Розширений алгоритм Евкліда реалізовано в ітеративній формі з початковими значеннями $r_0 = a$, $r_1 = b$, $x_0 = 1$, $x_1 = 0$, $y_0 = 0$, $y_1 = 1$. Якщо отриманий коефіцієнт x від'ємний, до нього додається модуль, доки значення не стане додатним.

```
/**
 * Розширений алгоритм Евкліда в ітеративній формі
 * (r0 = a, r1 = b; x0 = 1, x1 = 0; y0 = 0, y1 = 1).
 */
export function extendedGcd(a: bigint, b: bigint): ExtendedGcd {
  let [r0, r1] = [a, b];
  let [x0, x1] = [1n, 0n];
  let [y0, y1] = [0n, 1n];

  while (r1 !== 0n) {
    const q = r0 / r1; // ціла частина від ділення
    [r0, r1] = [r1, r0 - q * r1];
    [x0, x1] = [x1, x0 - q * x1];
    [y0, y1] = [y1, y0 - q * y1];
  }

  return { gcd: r0, x: x0, y: y0 };
}

/**
 * Число, обернене до `a` за модулем `m`: (a * inverse) mod m === 1.
 * Якщо коефіцієнт x від'ємний — додаємо модуль, доки не стане додатним.
 */
export function modInverse(a: bigint, m: bigint): bigint {
  const { gcd: g, x } = extendedGcd(a, m);
  if (g !== 1n) throw new Error(`Оберненого не існує: НСД(${a}, ${m}) = ${g}`);
  return ((x % m) + m) % m;
}
```

Лістинг 2 — Розширений алгоритм Евкліда та обернене за модулем (src/bigint-math.ts)

2.2. Хеш квадратичної згортки та ЕЦП

Перед підписанням бюлетень хешується спрощеною хеш-функцією квадратичної згортки $H_i = (H_{i-1} + M_i)^2 \bmod n$, де $H_0 = 0$, M_i — числове

представлення символу, n — модуль RSA підписувача. ЕЦП обчислюється як H^d mod n. Для перевірки ВК самостійно обчислює хеш отриманого тексту та порівнює його зі значенням H_c = ЕЦП^e mod n: збіг означає, що підпис справжній, а бюлетень не змінювався.

```
/**
 * Хеш квадратичної згортки: Hᵢ = (Hᵢ₋₁ + Mᵢ)² mod n, H₀ = 0,
 * де Mᵢ — числове представлення символу повідомлення.
 */
export function hashMessage(message: string, n: bigint): bigint {
  const bytes = new TextEncoder().encode(message);
  let h = 0n;
  for (const byte of bytes) h = ((h + BigInt(byte)) ** 2n) % n;
  return h;
}

/** Формування ЕЦП закритим ключем: ЕЦП = H^d mod n. */
export function sign(message: string, key: PrivateKey): bigint {
  return modPow(hashMessage(message, key.n), key.d, key.n);
}

/** Перевірка ЕЦП: порівнюємо H і H_c = ЕЦП^e mod n. */
export function verify(message: string, signature: bigint, key: PublicKey):
boolean {
  const expected = hashMessage(message, key.n);
  const recovered = modPow(signature, key.e, key.n);
  return expected === recovered;
}
```

Лістинг 3 — Хеш, формування та перевірка ЕЦП (src/rsa.ts)

2.3. Шифрування бюлетеня

Бюлетень разом з ЕЦП шифрується відкритим ключем ВК за формулою c = m^e mod n, а розшифровується закритим ключем за формулою m = c^d mod n. Оскільки повідомлення довше за модуль, воно розбивається на блоки: розмір блоку обирається так, щоб числове представлення блоку завжди було меншим за n. Перед кожним блоком додається маркер 0x01, щоб під час зворотного перетворення не втратити провідні нулі.

```
/**
 * Шифрування рядка відкритим ключем: c = m^e mod n для кожного блоку.
 * Перед блоком ставиться маркер 0x01, щоб не втратити провідні нулі.
 */
export function encrypt(message: string, key: PublicKey): bigint[] {
  const bytes = new TextEncoder().encode(message);
  const size = blockSize(key.n);
  const blocks: bigint[] = [];

  for (let offset = 0; offset < bytes.length; offset += size) {
    const chunk = bytes.subarray(offset, offset + size);
    const marked = new Uint8Array(chunk.length + 1);
    marked[0] = 0x01;
```

```
    marked.set(chunk, 1);
    blocks.push(modPow(bytesToBigInt(marked), key.e, key.n));
  }
  return blocks;
}

/** Розшифрування закритим ключем: m = c^d mod n. */
export function decrypt(blocks: readonly bigint[], key: PrivateKey): string {
  const chunks: Uint8Array[] = [];

  for (const block of blocks) {
    const marked = bigIntToBytes(modPow(block, key.d, key.n));
    if (marked[0] !== 0x01) throw new Error("Пошкоджений блок шифротексту");
    chunks.push(marked.subarray(1));
  }

  const total = chunks.reduce((sum, chunk) => sum + chunk.length, 0);
  const bytes = new Uint8Array(total);
  let offset = 0;
  for (const chunk of chunks) {
    bytes.set(chunk, offset);
    offset += chunk.length;
  }
  return new TextDecoder().decode(bytes);
}
```

Лістинг 4 — Блокове шифрування та розшифрування (src/rsa.ts)

2.4. Дії виборця

Виборець самостійно генерує власну пару ключів RSA, відкритий ключ передає ВК на етапі складання списку. Під час голосування він формує бюлетень (ПІБ, код кандидата, час формування), підписує його канонічний текст власним закритим ключем і шифрує пару «бюлетень + ЕЦП» відкритим ключем ВК. ПІБ відправника передається у відкритому вигляді, оскільки за умовою простого протоколу ВК має знати, чий бюлетень вона розшифровує.

```
/** Крок 2.1–2.3: сформувати бюлетень, захешувати та підписати закритим ключем.
*/
fillAndSign(candidateId: string, issuedAt = new Date().toISOString()):
SignedBallot {
  const ballot: Ballot = { voterName: this.name, candidateId, issuedAt };
  const text = serializeBallot(ballot);
  return { ballot: text, signature: sign(text, this.keys.privateKey) };
}

/** Крок 2.4: запечатати підписаний бюлетень відкритим ключем ВК. */
seal(signed: SignedBallot, commissionKey: PublicKey): Envelope {
  return { senderName: this.name, cipher: sealFor(signed, commissionKey) };
}
```

Лістинг 5 — Формування, підписання та шифрування бюлетеня (src/voter.ts)

2.5. Перевірки виборчої комісії

ВК розшифровує отриманий конверт власним закритим ключем і послідовно виконує перевірки: наявність відправника у списку допущених виборців; справжність ЕЦП відкритим ключем, заздалегідь закріпленим за цим виборцем; відповідність ПІБ у бюлетені ПІБ відправника; відсутність позначки про попереднє голосування; коректність заповнення бюлетеня. Бюлетень приймається лише тоді, коли всі перевірки пройдено, після чого у списку виборців з'являється позначка про голосування, а голос додається до підрахунку.

```
/** Крок 3: розшифрування бюлетеня та всі передбачені протоколом перевірки. */
receive(envelope: Envelope): ProcessingResult {
  const opened = this.open(envelope);
  if (!opened.ok) return this.reject(envelope.senderName, "UNDECRYPTABLE",
opened.error);

  const { ballot, signature, signedText } = opened;
  this.note("decrypt", true, `бюлетень розшифровано: кандидат
«${ballot.candidateId}»`);

  const voter = this.voters.get(envelope.senderName);
  if (!voter) {
    this.note("inList", false, `«${envelope.senderName}» відсутній у списку
виборців`);
    return this.reject(envelope.senderName, "NOT_IN_LIST");
  }
  this.note("inList", true, `«${voter.name}» є у списку допущених виборців`);

  const signatureValid = verify(signedText, signature, voter.publicKey);
  this.note(
    "signature",
    signatureValid,
    signatureValid ? "H = H_c, ЕЦП справжній" : "H ≠ H_c, ЕЦП недійсний",
  );
  if (!signatureValid) return this.reject(envelope.senderName,
"BAD_SIGNATURE");

  const identityMatches = ballot.voterName === envelope.senderName;
  this.note(
    "identity",
    identityMatches,
    identityMatches
      ? "ПІБ у бюлетені збігається з відправником"
      : `у бюлетені «${ballot.voterName}», а надіслав
«${envelope.senderName}»`,
  );
  if (!identityMatches) return this.reject(envelope.senderName,
"IDENTITY_MISMATCH");

  if (voter.hasVoted) {
    this.note("notVotedYet", false, `«${voter.name}» уже має позначку про
голосування`);
    return this.reject(envelope.senderName, "ALREADY_VOTED");
  }
  this.note("notVotedYet", true, `позначки про голосування «${voter.name}»
немає`);
```

```
  const candidate = this.candidates.find((item) => item.id ===
ballot.candidateId);
  if (!candidate) {
    this.note("ballotValid", false, `кандидата «${ballot.candidateId}» немає у
списку`);
    return this.reject(envelope.senderName, "INVALID_BALLOT");
  }
  this.note("ballotValid", true, `голос за кандидата «${candidate.name}»`);

  voter.hasVoted = true; // фіксація факту голосування
  this.tally.set(candidate.id, (this.tally.get(candidate.id) ?? 0) + 1);
  this.acceptedCount += 1;

  return { status: "accepted", voterName: voter.name, candidateId: candidate.id
};
}
```

Лістинг 6 — Обробка бюлетеня виборчою комісією (src/commission.ts)

2.6. Підрахунок голосів і публікація результатів

Після опрацювання всіх бюлетенів ВК публікує результати: кількість голосів за кожного кандидата, кількість прийнятих бюлетенів і відхилених спроб, явку та переможця. Якщо максимальну кількість голосів набрали декілька кандидатів, результат позначається як нічия — програма не обирає переможця довільно, а фіксує рівномірний розподіл голосів.

```
/** Крок 4: підрахунок голосів і публікація результатів. */
publishResults(): ElectionResults {
  const max = Math.max(0, ...this.tally.values());
  const leaders = this.candidates.filter((candidate) =>
this.tally.get(candidate.id) === max);

  return {
    tally: new Map(this.tally),
    accepted: this.acceptedCount,
    rejected: this.rejectedCount,
    turnout: {
      voted: this.voterList.filter((voter) => voter.hasVoted).length,
      registered: this.voters.size,
    },
    leaders: max === 0 ? [] : leaders,
    isTie: max > 0 && leaders.length > 1,
  };
}
```

Лістинг 7 — Публікація результатів голосування (src/commission.ts)

3. Результати виконання протоколу та передбачених перевірок

Демонстраційний сценарій запускається командою npm start. У ньому беруть участь два кандидати та п’ять виборців; для кожної перевірки ВК передбачено як позитивний, так і негативний випадок. Зведення сценаріїв наведено в таблиці 2.

Сценарій	**Що перевіряється**	**Результат**
Коректний бюлетень (5 виборців)	усі перевірки ВК проходять	бюлетень прийнято, голос зараховано, виборця позначено
Повторне голосування	перевірка позначки про голосування	відхилено: ALREADY_VOTED
Стороння особа	наявність у списку виборців	відхилено: NOT_IN_LIST
Голосування від чужого імені	перевірка ЕЦП ключем виборця	відхилено: BAD_SIGNATURE
Підміна бюлетеня в дорозі	цілісність: H ≠ H_c	відхилено: BAD_SIGNATURE
Зіпсований бюлетень (К9)	коректність заповнення	відхилено: INVALID_BALLOT
Бюлетень, зашифрований чужим ключем	розшифрування закритим ключем ВК	відхилено: UNDECRYPTABLE
Рівний розподіл голосів (2:2)	публікація результатів	зафіксовано нічию, переможця не визначено

Таблиця 2 — Перевірені сценарії роботи протоколу

На рисунку 1 показано перший крок протоколу: кожен з п’яти виборців згенерував власну пару ключів довжиною 512 біт, ВК сформувала список допущених виборців із їх відкритими ключами, список кандидатів та опублікувала власний відкритий ключ для шифрування бюлетенів.

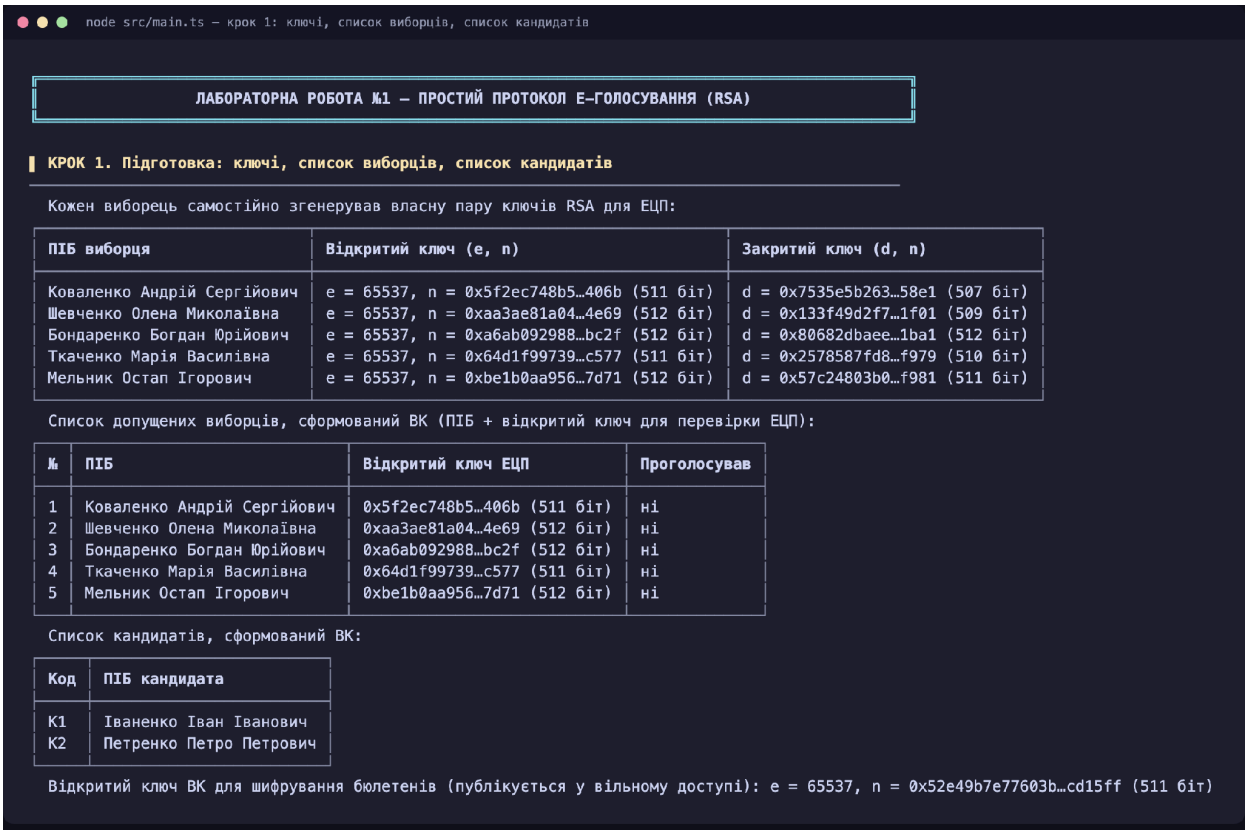


Рисунок 1 — Крок 1: генерація ключів, список виборців і список кандидатів

На рисунку 2 наведено позитивний сценарій: показано текст бюлетеня, його хеш, значення ЕЦП та блоки шифротексту, а також послідовне проходження всіх перевірок ВК і зарахування голосу.

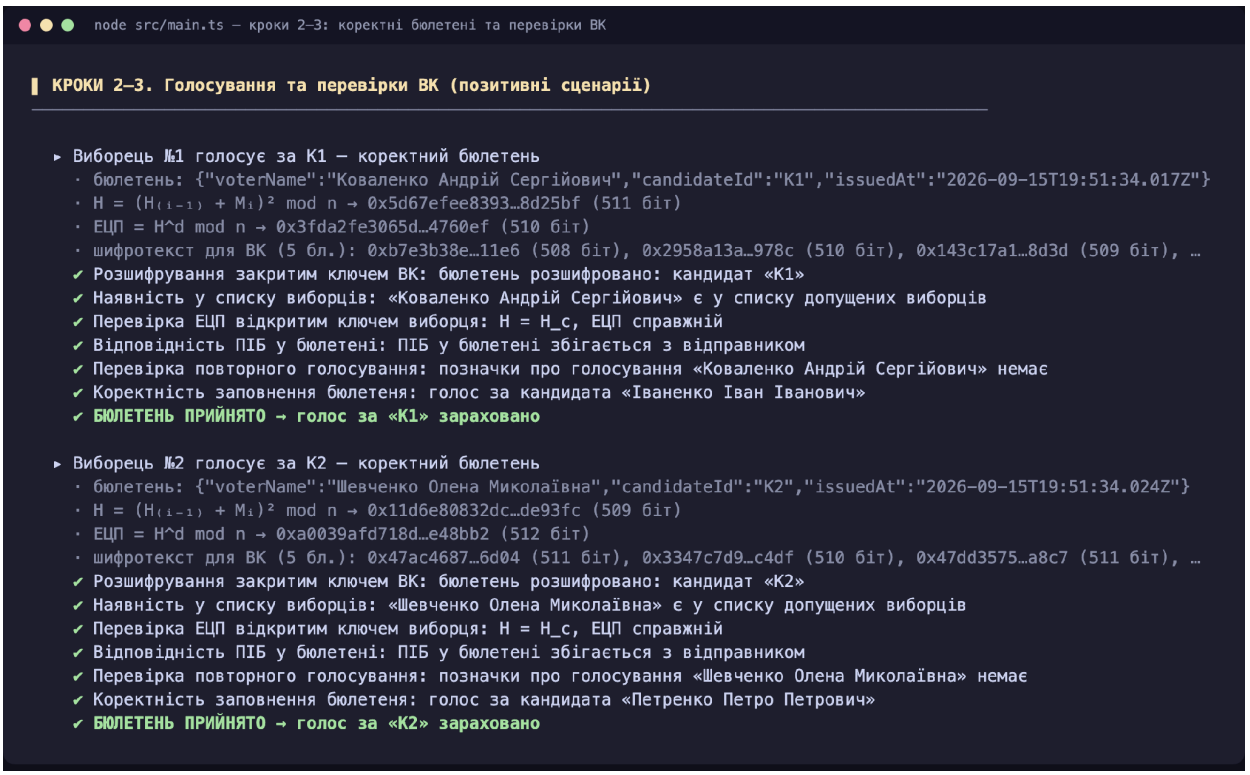


Рисунок 2 — Кроки 2–3: формування, підписання, шифрування та прийняття бюлетеня

Рисунок 3 демонструє негативні сценарії 1–3. Повторне голосування зупиняється перевіркою позначки у списку виборців; бюлетень сторонньої особи відхиляється на перевірці списку; спроба проголосувати від чужого імені виявляється перевіркою ЕЦП, оскільки зловмисник не має закритого ключа жертви.

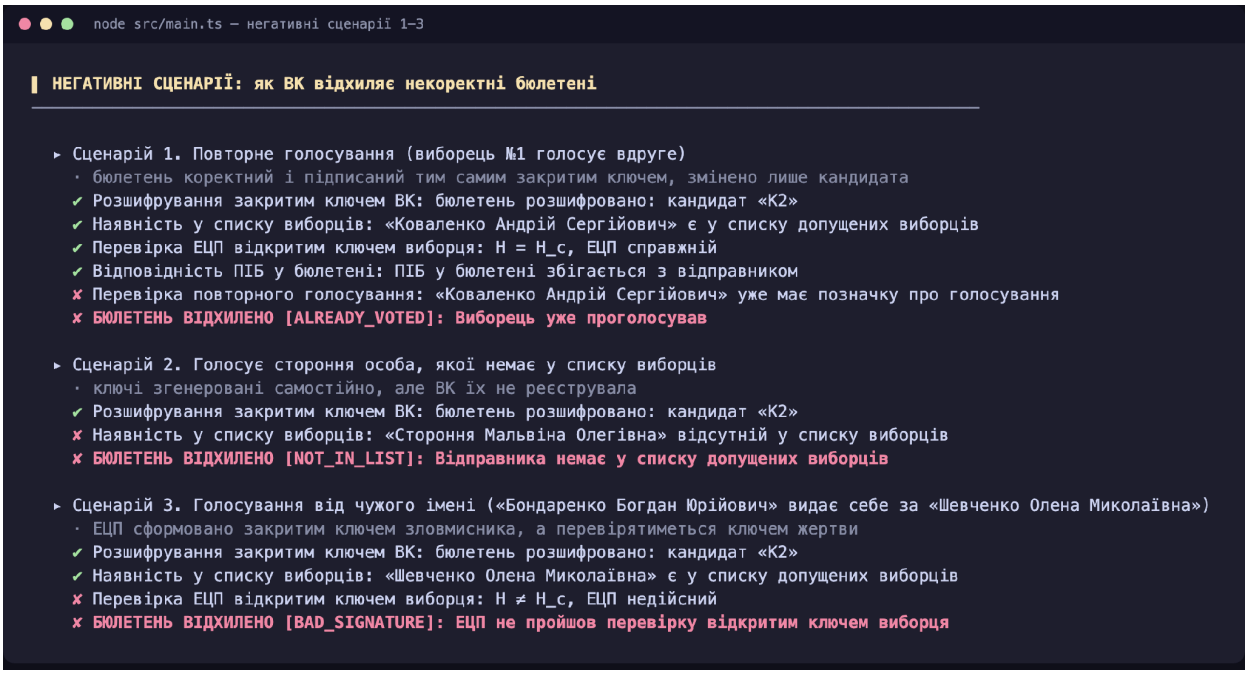


Рисунок 3 — Негативні сценарії: повторне голосування, стороння особа, чуже ім’я

Рисунок 4 демонструє негативні сценарії 4–5. Підміна бюлетеня по дорозі до ВК можлива технічно (відкритий ключ ВК доступний усім), але змінений текст більше не відповідає ЕЦП, тому бюлетень відхиляється. У п’ятому сценарії ЕЦП правильний, проте бюлетень заповнений некоректно: указаного кандидата не існує. Це підтверджує, що ЕЦП гарантує авторство та цілісність, але не коректність заповнення.

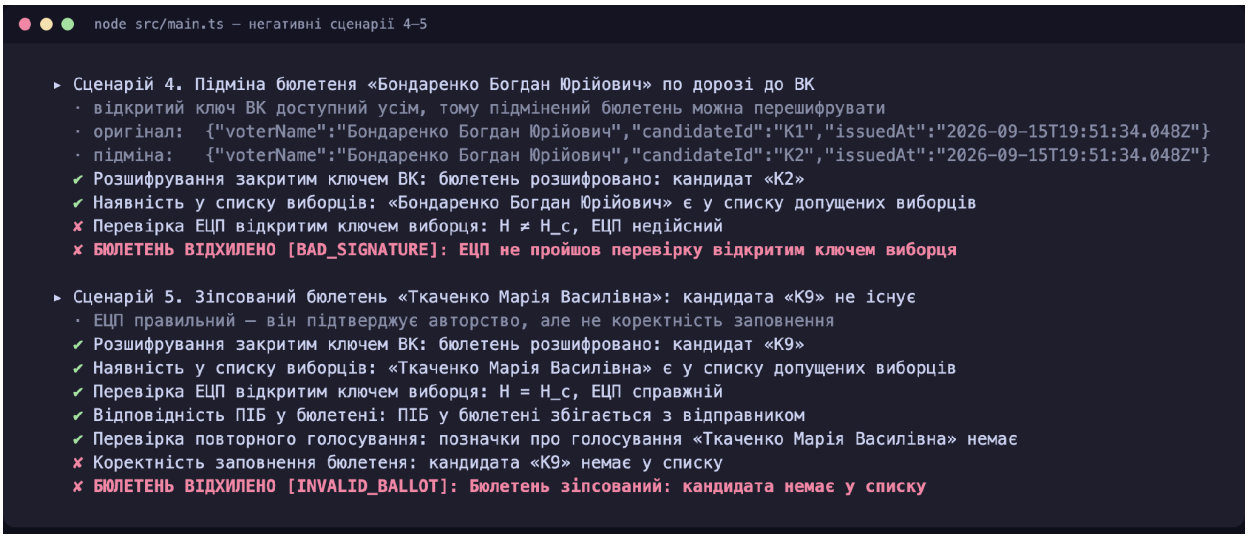


Рисунок 4 — Негативні сценарії: підміна бюлетеня та зіпсований бюлетень

На рисунку 5 показано четвертий крок протоколу — підрахунок голосів і публікацію результатів: прийнято 5 бюлетенів, відхилено 5 некоректних спроб, підсумок 3:2 на користь першого кандидата, усі п’ятеро виборців мають позначку про голосування.

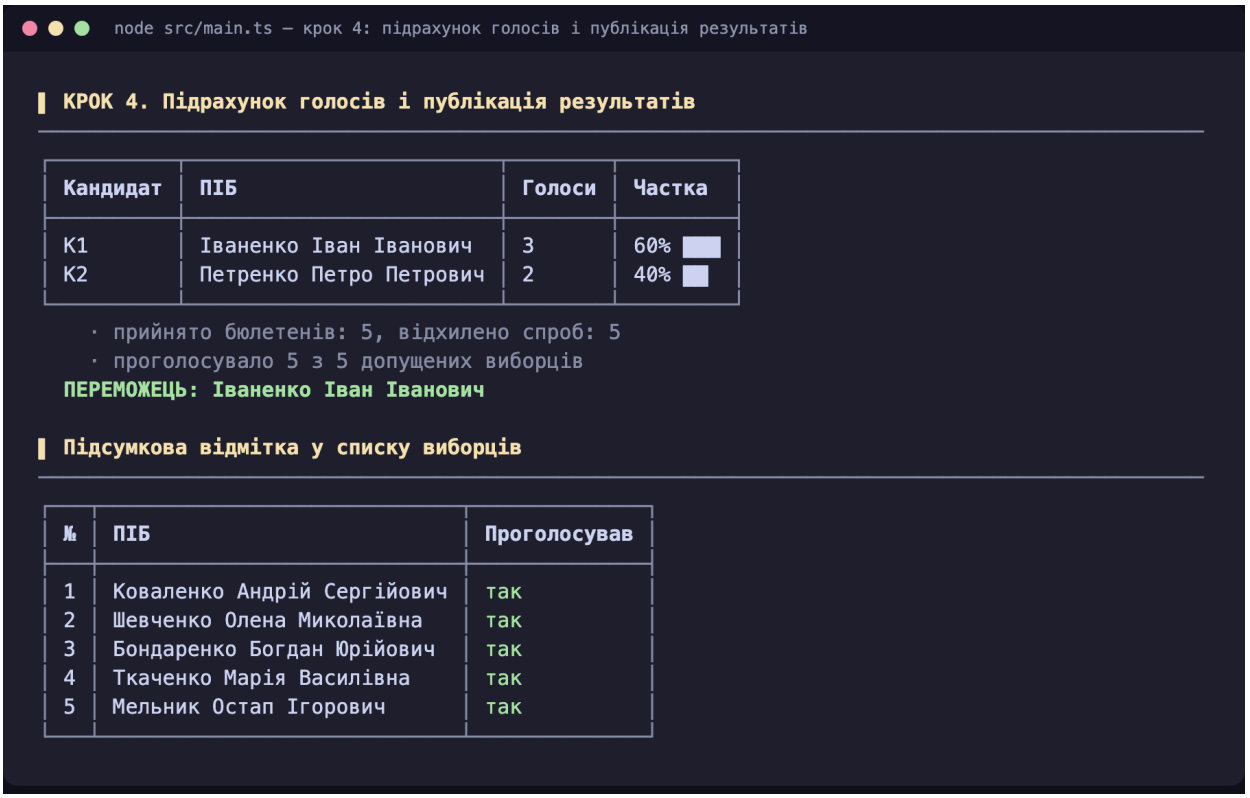


Рисунок 5 — Крок 4: підрахунок голосів, публікація результатів та підсумковий список виборців

Рисунок 6 ілюструє врахування рівномірного розподілу голосів: за результатами окремої сесії з чотирма виборцями голоси розподілилися 2:2, тому програма фіксує нічию. Нижче наведено зведення відповідності реалізованого протоколу вимогам до ідеального протоколу електронного голосування.

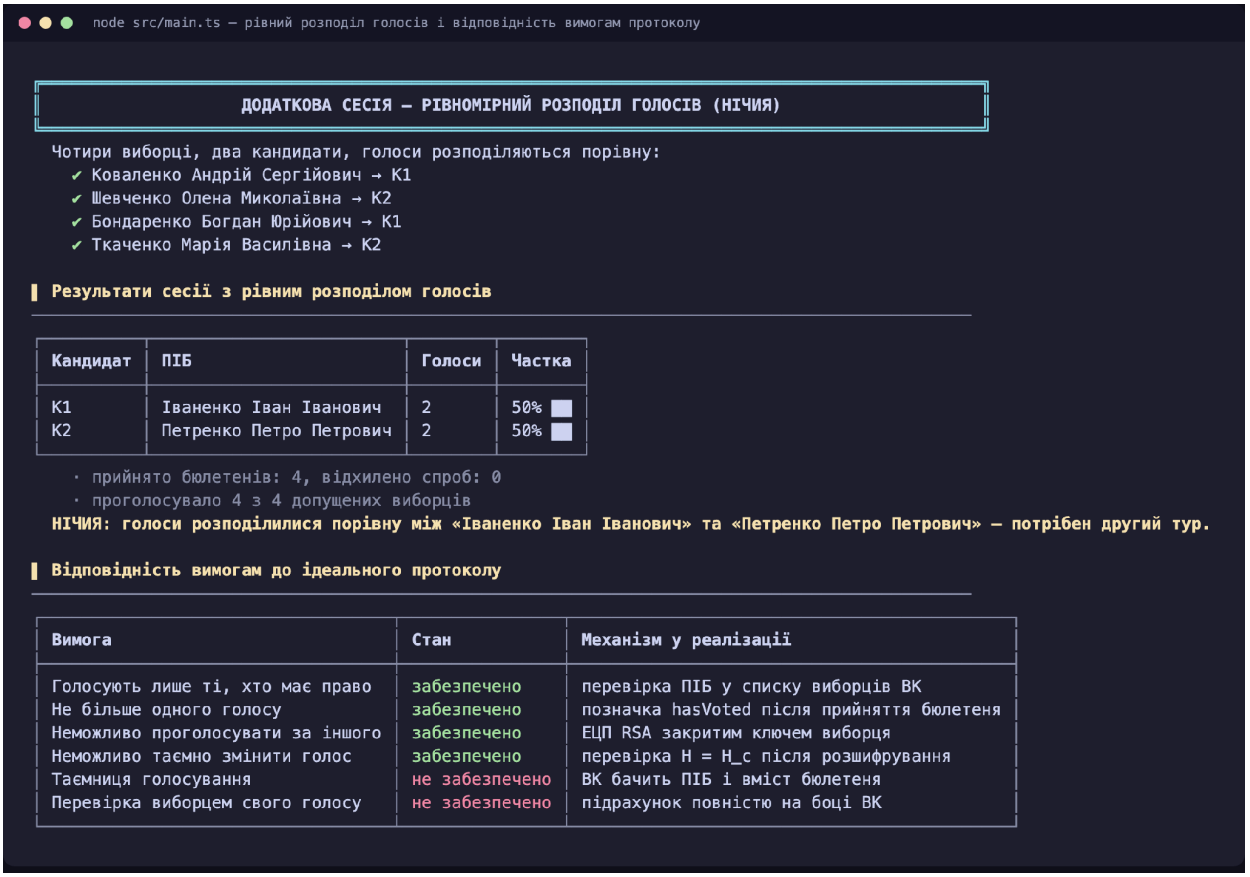


Рисунок 6 — Рівномірний розподіл голосів (нічия) та відповідність вимогам протоколу

4. Тестування

Окрім демонстраційного сценарію, реалізацію покрито 21 модульним тестом (команда npm test). Тести перевіряють коректність арифметики (розширений алгоритм Евкліда, обернене за модулем, тест простоти, модульне піднесення до степеня), узгодженість ключів RSA ($e \cdot d \equiv 1 \bmod \varphi(n)$), оборотність шифрування для кириличного тексту, чутливість хешу до зміни символу, перевірку ЕЦП чужим ключем, а також усі позитивні та негативні сценарії протоколу, наведені в таблиці 2.



Рисунок 7 — Результати запуску модульних тестів

5. Висновок

У ході виконання лабораторної роботи програмно реалізовано всі кроки простого протоколу електронного голосування для двох кандидатів і п’яти виборців. Реалізовано самостійну генерацію виборцями пар ключів RSA, формування ВК списку виборців з їх відкритими ключами та списку кандидатів, публікацію відкритого ключа ВК, формування, підписання та шифрування бюлетеня, його розшифрування, повний набір перевірок, фіксацію факту голосування, підрахунок голосів і публікацію результатів, включно з випадком рівномірного розподілу голосів.

Експериментально підтверджено, що простий протокол забезпечує чотири вимоги до ідеального протоколу: голосувати можуть лише ті, хто має

право (перевірка за списком); кожен голосує не більше одного разу (позначка у списку); ніхто не може проголосувати замість іншого та непомітно змінити чужий голос (ЕЦП на закритому ключі виборця).

Водночас протокол не забезпечує таємниці голосування: ВК знає ПІБ виборця та після розшифрування бачить вміст його бюлетеня. Виборець також не має незалежного способу перевірити, що його голос врахований у кінцевому результаті, оскільки приймання бюлетенів і підрахунок повністю контролює ВК. Тому коректність результату залежить від доброчесності виборчої комісії, і для реальних виборів простий протокол потребує доповнення механізмами анонімізації (наприклад, сліпим підписом) та перевірюваності результату.